UNIT II

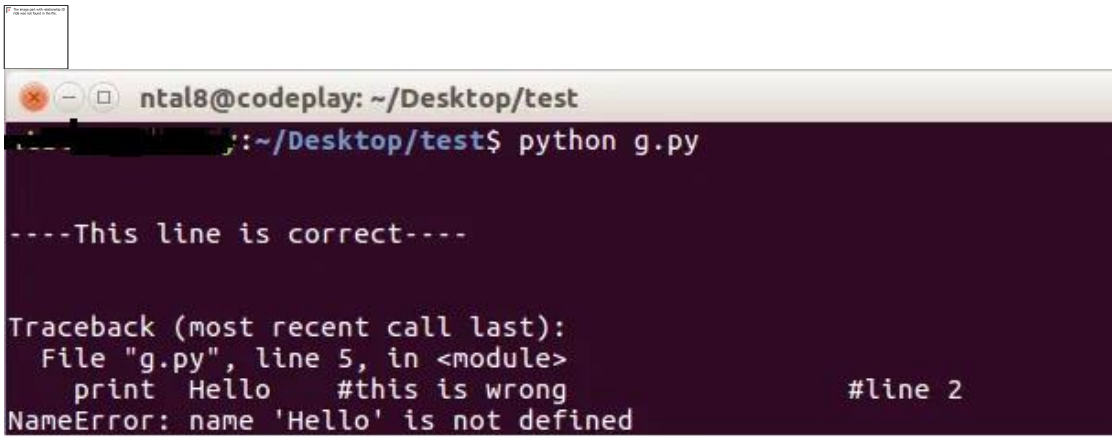
Interpreted and Interactive Python

Interpreted Python

Unlike C/C++ etc, Python is an **interpreted object-oriented programming language**. By interpreted it is meant that each time a program is run the interpreter checks through the code for errors and then interprets the instructions into machine-readable bytecode.

An interpreter is a translator in computer's language which translates the given code line-by-line in machine readable bytecodes. And if any error is encounterd it stops the translation until the error is fixed. Unlike C language, which is a **compiled programming language**. The compiler translates the whole code in one-go rather than line-by-line. This is the reason why in C language, all the errors are listed during compilation only.

```
Demonstrating interpreted python

print "\n\n----This line is correct----\n\n"   #line1

print Hello     #this is wrong                  #line2
```



In the above illustration, you can see that **line 1** was syntactically correct and hence got successfully executed. Whereas, in **line 2** we had a syntax error. Hence the interpreter stopped executing the above script at **line 2**. This is not valid in the case of a compiled programming language.

The illustration below is a C++ program, see that although **line 1** and **line 2** were correct than also the program reports an error, due to an error on **line 3**. This is the basic **difference between interpreted language and compiled language.**

```
// Demonstrating Compiled language

#include<iostream>

using namespace std;

int main(){

        cout<<"---This is correct line---";       // line 1

        cout<<"---This line is also correct---"; // line 2

        cout "this is Incorrect Line"             // line 3

        return 0;}
```

```
:~/Desktop/test$ g++ hello.cpp -o c_file
hello.cpp: In function 'int main()':
hello.cpp:10:7: error: expected ';' before string constant
  cout "this is Incorrect Line"             // line 3
       ^
```

Interactive Python

Python is interactive. When a Python statement is entered, and is followed by the Return key, if appropriate, the result will be printed on the screen, immediately, in the next line. This is particularly advantageous in the debugging process. In interactive mode of operation, Python is used in a similar way as the Unix command line or the terminal.

The interactive Python shell looks like:

```
Python 2.7.12 (default, Nov 19 2016, 06:48:10)
[GCC 5.4.0 20160609] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Interactive Python is very much helpful for the debugging purpose. It simply returns the >>> prompt or the corresponding output of the statement if appropriate and returns **error** for incorrect statements. In this way if you have any doubts like: whether a syntax is correct, whether the module you are importing exists or anything like that, you can be sure within seconds using Python interactive mode.

```
Python 2.7.12 (default, Nov 19 2016, 06:48:10)
[GCC 5.4.0 20160609] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> print "Hello world"
Hello world
>>> print "This line is correct"
This line is correct
>>>
>>> import socket
>>>
>>> import Image
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/usr/local/lib/python2.7/dist-packages/PIL/Image.py", line 27, in <modul
e>
    from . import VERSION, PILLOW_VERSION, _plugins
ValueError: Attempted relative import in non-package
>>>
>>> print error
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'error' is not defined
>>>
```

How to Run Python Program?

A Python script, a plain text file with .py or .pyw extension, is run by the Python interpreter. The interpreter, necessary for running Python code, operates in two modes: **executing scripts/modules** or **running code in interactive sessions**.

Different Ways to Run Python Program

There are various ways to run a Python Program, let us see them -

1. The Python Interactive Mode

This is the most frequently used way to run a Python code. Here basically we do the following:

1. Open command prompt / terminal in your system
2. Enter the command - python or python3 (based on python version installed in your system)
3. If we see these : >>> then, we are into our python prompt. It will look something like this:

```
C:\Users\Deepa>pythonPython 3.9.2
(tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC
v.1928 64 bit (AMD64)] on win32Type "help",
"copyright", "credits" or "license" for more information.
>>>
```

Now we can start writing our code here.

Example 1:

```
>>> print("Hello World!")
Hello World!
```

Example 2:

```
>>> a = 10
>>> b = 20
```

```
>>> sum = a + b
>>> print("The sum of a & b is : ",sum)
The sum of a & b is :  30
>>> 5 + 10
>>> 15
```

Example 3:

```
>>> for i in range(5):
...     print(i*2)
...
0
2
4
6
8
```

Pros:

1. Best for testing every single line of code written.
2. In this approach, each line of code is evaluated and executed immediately as soon as we hit ↵ **Enter**.
3. Great deveopment tool for experimentation of Python code on the way.

Cons:

1. The code is gone as soon as we close the terminal(or the interactive session)
2. Not user friendly way to write long code.

To close the interactive session we can:

1. Type quit() or exit()
2. Press ctrl + z and hit ↵ **Enter** for windows. For linux, we can press ctrl+d.

2. Using the Python Command Line

Follow the below steps to run a Python Code through command line -

1. Write your Python code into any text editor.
2. Save it with the extension .py into a suitable directory

```
# Save it with any suitable file name(suppose hello.py)
print("Hello World!!")
```

1. Open the terminal or command prompt.
2. Type python/python3 (based on installation) followed by the file name.py --

```
python YourFileName.py
```

Example 1: Saved with hello.py

```
print("Hello World!!")
```

OUTPUT:

```
C:\Users\deepa>python hello.py
Hello World!!
```

Example 2: Saved with factorial.py

```
factorial = 1
for i in range(1,6):
        factorial = factorial*i
print("The factorial of 5 is = ",factorial)
```

OUTPUT:

```
C:\Users\deepa>python factorial.py
The factorial of 5 is = 120
```

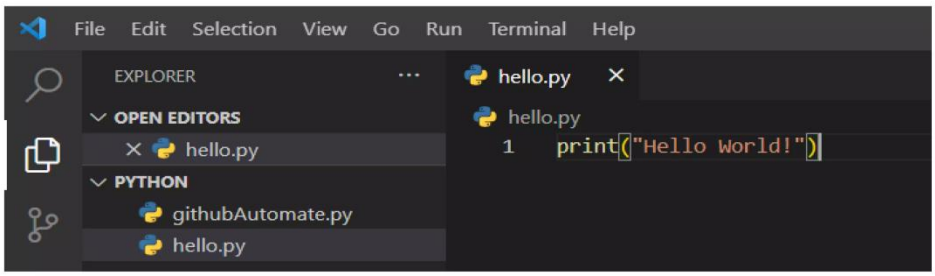
Note: If it doesn't runs, then re-check your Python installation path & the place you saved your Python file.

3. Run Python on Text Editor

If you want to run your Python code into a text editor -- suppose VS Code, then follow the below steps:

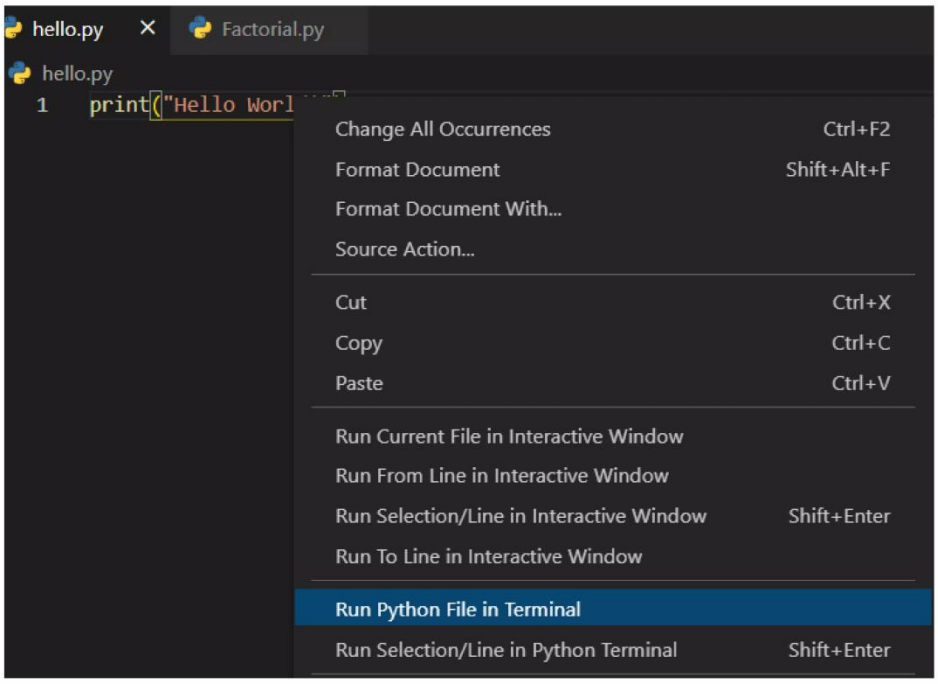
1. From extensions section, find and install 'Python' extension.

2. Restart your VS code to make sure the extension is installed properly
3. Create a new file, type your Python code and save it.

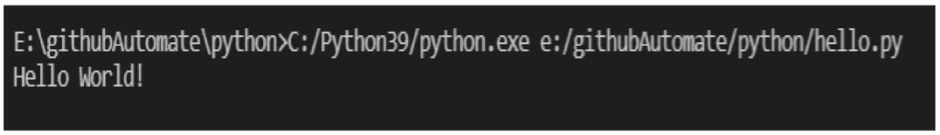


4. Run it using the VS code's terminal by typing the command "py hello.py"

5. Or, Right Click -> Run Python file in terminal



OUTPUT:

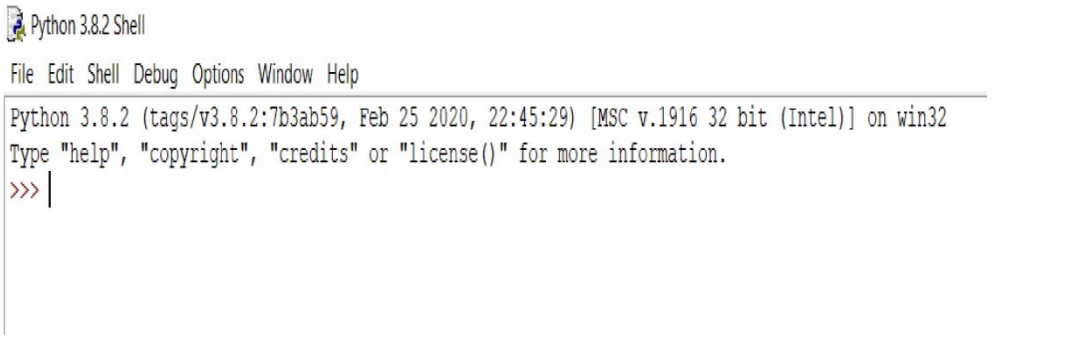


4. Run Python on IDLE

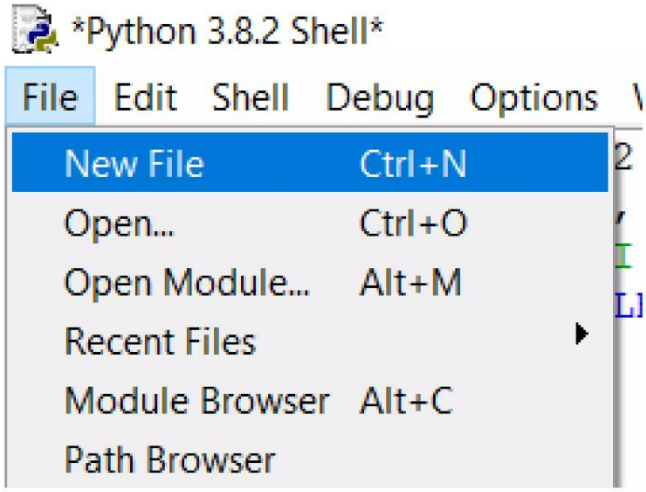
Python installation comes with an **Integrated Development and Learning Environment**, which is popularly known as IDLE. Though Python IDLE are by-default included with Python installations on Windows and Mac, if you're a Linux user, then you can easily download Python IDLE using your package manager.

Steps to run a python program on IDLE:

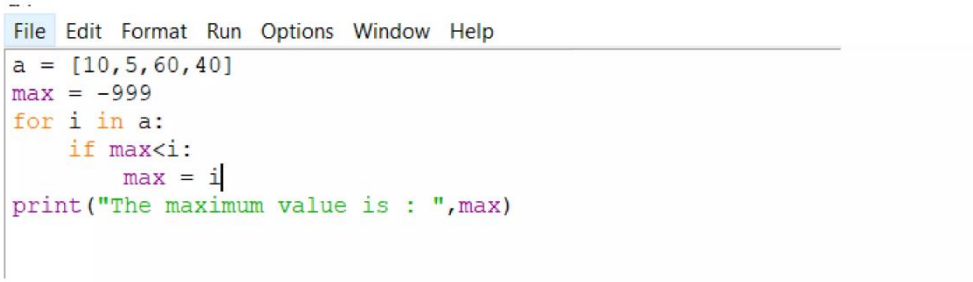
1. Open the Python IDLE(navigate to the path where Python is installed, open IDLE(python version))
2. The shell is the default mode of operation for Python IDLE. So, the first thing you will see is the Python Shell --



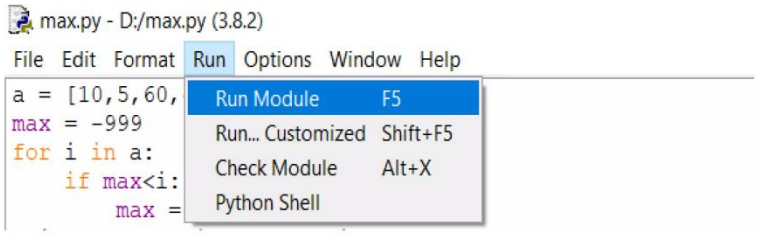
3. You can write any code here and press enter to execute
4. To run a Python script you can go to **File -> New File**



5. Now, write your code and save the file --

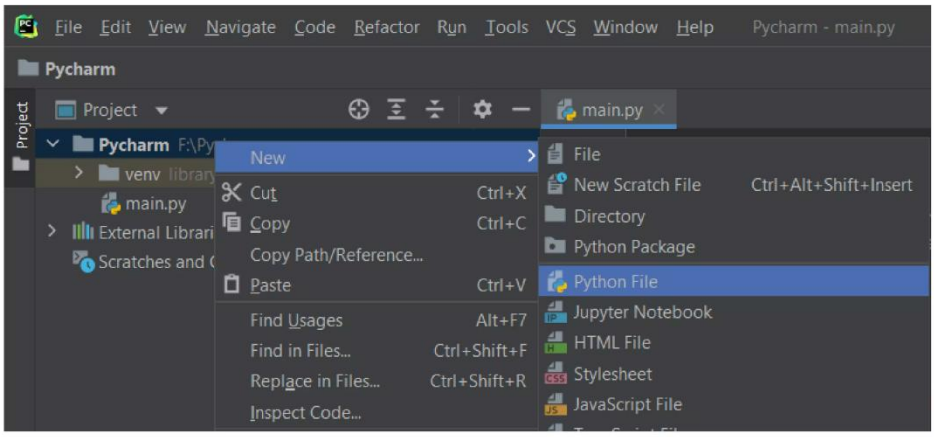


6. Once saved, you can run your code from **Run -> Run Module** Or simply by **pressing F5** --

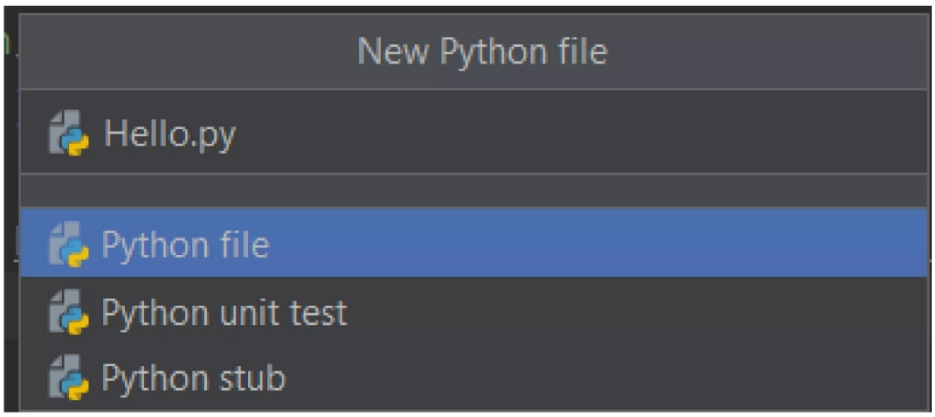


5. Run Python On IDE (PyCharm)

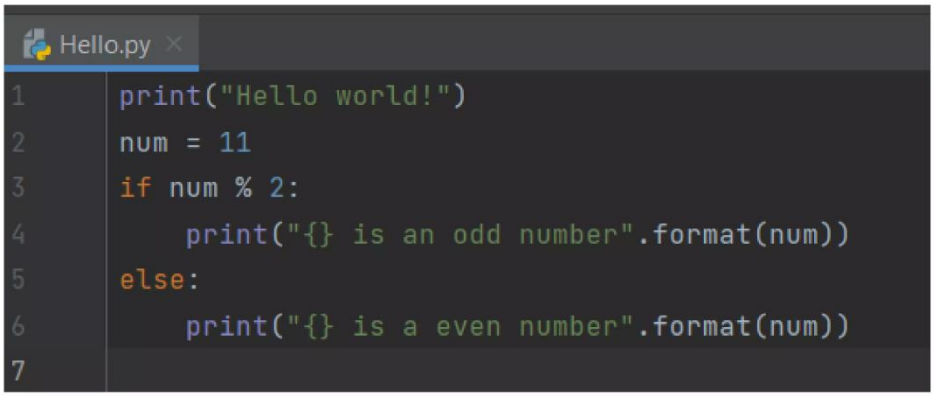
1. In the Project tool window, select the project root, right-click it, and select File -> New -> Python File



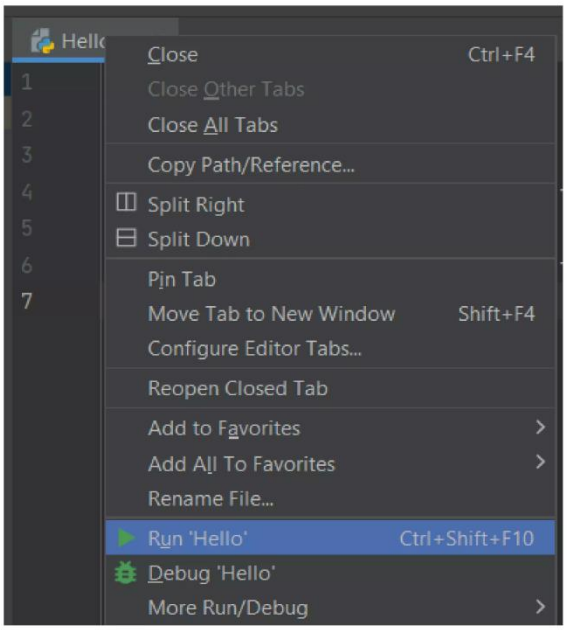
2. Then you will see a prompt, select the option Python File from the menu, and then type the new filename.



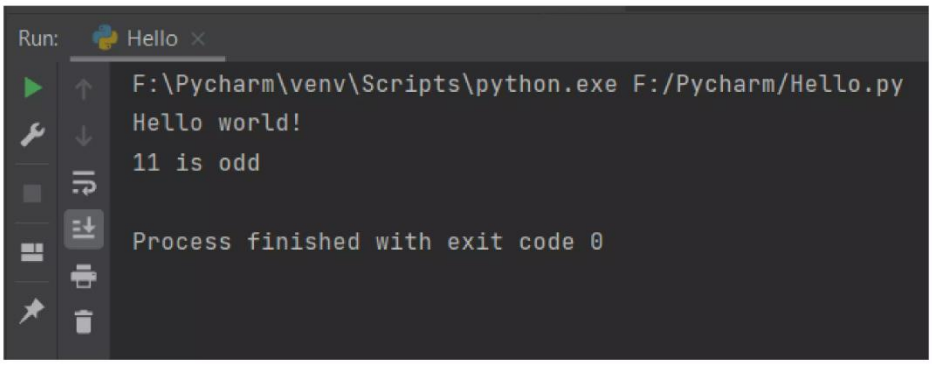
3. Write your code in the Python file you have just created.



4. To run this file, **Right click on the file name -> Run filename**



5. PyCharm executes your code in the Run tool window. Check your output there:



Variables in Python

Variables in Python are fundamental building blocks, serving as containers for storing data values. In Python, memory space is allocated automatically without the need for explicit declaration when a value is assigned to a variable. The assignment of values to variables is done using the equal sign (=).

Example of Variable in Python

Python is dynamically-typed, which means you don't need to declare the type of a variable explicitly. Here's an example to illustrate how variables work in Python:

```
# Assigning a value to a variable
score = 100
# Assigning a string to another variable
player_name = "Alex"
# Assigning a list to another variable
items_collected = ['sword', 'shield', 'potion']
# Printing the values stored in variables
print("Score:", score)
print("Player Name:", player_name)
print("Items Collected:", items_collected)
```

Output:

```
Score: 100
Player Name: Alex
Items Collected: ['sword', 'shield', 'potion']
```

Variables Assignment

Python simplifies variable assignment, making it easy and intuitive for developers.

Creating Variables

In Python, creating a variable is as simple as assigning it a value. Python is dynamically typed, so you don't need to declare the type. The variable is created the moment you first assign a value to it.

```
x = 5
name = "John"
print(name)
```

Output:

```
John
```

Python Redeclaring Variables

You can redeclare a variable in Python by assigning a new value to it. This can even change the variable's type, due to Python's dynamic typing.

```
x = 5
x = "Sara"
print(x)
```

Output:

```
Sara
```

Initially, x is an integer. After redeclaration, x becomes a string.

Python Assign Values to Multiple Variables

Python allows assigning values to multiple variables in one line, making your code concise and readable.

```
a = b = c = 5
print(a)
print(b)
print(c)
```

Output:

```
5
5
5
```

Assigning Different Values to Multiple Variables

You can assign different types of values to multiple variables in a single statement in Python.

```
a, b, c = 5, 3.2, "Hello"
print(a)
print(b)
print(c)
```

Output:

```
5
3.2
Hello
```

Here, a becomes an integer (5), b a float (3.2), and c a string ("Hello").

Can We Use the Same Name for Different Types?

Yes, in Python, we can use the same variable name for different types. Due to Python's dynamic typing, the type of a variable is inferred from the value assigned, and the type can change as you reassign different values.

```
x = 100       # x is an integer
x = "Python"  # Now x is a string
print(x)
```

Output:

```
Python
```

+ Operator with Variables

The + operator in Python can be used with variables for addition or concatenation, depending on the data type.

- For numbers, it performs addition:

```
a = 10
b = 20
print(a + b)
```

Output:

```
30
```

- For strings, it performs concatenation:

```
first_name = "John"
last_name = "Doe"
print(first_name + " " + last_name)
```

Output:

```
John Doe
```

Using the + operator between different types (e.g., a string and an integer) without explicit conversion will result in a TypeError.

Global and Local Python Variables

In Python, variables' scope—where they can be accessed—is determined by where they are defined. The two main types of variable scope are global and local.

Global Variables

Global variables are declared outside of functions or in the global scope, meaning they can be accessed throughout your code, including inside functions. They are useful when multiple functions need to access the same data.

```
x = "global"
def function():
        print("X inside:", x)
function()
print("X outside:", x)
```

Output:

```
X inside: global
X outside: global
```

In this example, x is a global variable accessible both inside and outside of the function().

Local Variables

Local variables are declared inside a function and can only be used within that function's scope of variables in python. They are created when the function starts and destroyed when the function ends.

```
def function():
        y = "local"
        print("Y inside function:", y)
function()
# print(y)  # This would raise an error because y is not
accessible here.
```

Output:

```
Y inside function: local
```

Here, y is a local variable, only accessible inside function().

Global keyword in Python

The global keyword in Python plays a pivotal role in modifying global variables from within a function's scope. It's used to declare that a variable inside a function is global, not local, allowing you to update the global variable's value from within the function.

Example

Without the global keyword, any variable assigned a value within a function is assumed to be local unless explicitly declared as global.

```
x = 10
def change_global_x():
        global x  # Declare x as global
        x = 20    # Modify the global variable
print("Before changing:", x)
change_global_x()
print("After changing:", x)
```

Output:

```
Before changing: 10
```

```
After changing: 20
```

In this example, the global keyword allows the change_global_x function to modify the global variable x.

Python Data Types

In Python, data types are classifications that specify the kind of values a variable can hold. There are several built-in data types in Python, broadly categorized into Numeric, Sequence Types, Mappings, Sets, and Boolean.

Introduction to Data types in Python

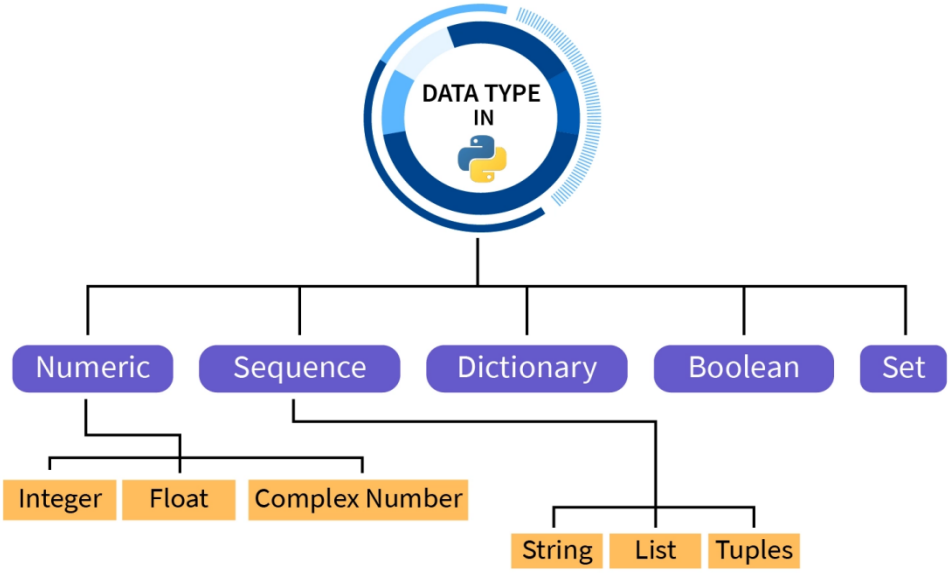
In programming, we often work with a variety of values: sentences, numbers, collections of items, and more. For example, a student's marks, represented by a number, would be considered an integer data type in Python, while a student's name, which is a sequence of characters, would be a string data type.

In Python, every value is associated with a data type because Python is an object-oriented language where everything is treated as an object.

Before proceeding further it's important to know that data types are also classified on the basis of their **mutability**. Mutable data types can be modified after creation, whereas Immutable data types can't be modified after creation.

Data Types in Python

Python is an object-oriented high-level programming language that offers a vast variety of data types helping in the implementation of various applications.



Following are the data types in Python

- Numeric Data types
- Sequence Data types
- Dictionaries in Python
- Booleans in Python
- Sets in Python

1. Numeric Data types in Python

Numeric data types in Python represent numbers and are categorized into three types:

- **Integers in Python**

Integers, as you know in mathematics are numbers without any fractional part. They can be 0, positive or negative. There is no limit to how long an integer can be in Python. They are represented by int class.

Syntax:

Integer numbers are of int type. It is just written as a number. Here's an example of Integer data type:

```
# Printing an integer in Python
num = 5
```

```
print("number =", num)
```

Output:

```
number = 5
```

- **Floating Point numbers in Python**

Floating point numbers (float) are real numbers with floating point representation, they are specified by a decimal point. They are represented by the float class.

Syntax:

Floating point numbers are of float type. It is written as a number with a decimal point. Here's an example of Float data type:

```
# Printing a floating point number in Python
num = 5.55
print("number =", num)
```

Output:

```
number = 5.55
```

- **Complex numbers in Python**

Complex numbers in python are specified as (real part) + (imaginary part)j. They are represented by the complex class.

Syntax:

Complex numbers are of complex type. They are written in the form a+bj in which a is the real value and b is the imaginary value, where j represents the imaginary part. Here's an example of a Complex data type:

```
# Printing a complex number in Python
num = 5 + 5j
print("number =", num)
```

Output

```
number = (5+5j)
```

Note: In Python, the type() function is used to determine the data type of a value or a variable. To know more about type() function.

```
# Program to demonstrate numerical values in Python.
x = 10
y = 10.0
z = 10 + 10j
print("Data type of", x, ":", type(x))
print("Data type of", y, ":", type(y))
print("Data type of", z, ":", type(z))
```

Output:

```
Data type of 10 : <class 'int'>
Data type of 10.0 : <class 'float'>
Data type of (10+10j) : <class 'complex'>
```

In the above example, we are using the type() function to show the type of data. It can be seen that "10" is an int type, "10.0" is a float type, and "10+10j" is a complex type.

2. Sequence Data types in Python

Sequence data types in Python are an ordered collection of similar or different values. They are also called container data types as they usually contain more than one value. We can access elements of a sequence data type by indexing. String, list, and tuple are the different types of containers used to store the data in a sequential manner.

- **Strings in Python**

A string can be defined as a sequence of characters enclosed in single, double, or triple quotation marks. While in most cases single and double quotation marks are interchangeable. Triple quotation marks are used for

multi-line strings. It is an immutable data type, i.e. its values cannot be updated. String is represented by string class.

Syntax:

Strings are of string type, and are represented by enclosing in quotation marks.

```
# Python program to demonstrate strings.# String with
single quotes
x = 'Scaler'
print("x = ", x)
print("The data type of x:", type(x))
print()
# String with double quotesy = "Scaler"
print("y =", y)
print("The data type of y:", type(y))
print()
# String with triple quotes (Multi-line strings)
z = '''Triple quotes are used
        for multi-line strings'''
print("z =", z)
print("The data type of z:", type(z))
```

Output:

```
 x = Scaler
The data type of x: <class 'str'>
y = Scaler
The data type of y: <class 'str'>
z = Triple quotes are used
        for multi-line strings
```

```
The data type of z: <class 'str'>
```

In the above example, we are demonstrating Python string, using single, double, and triple quotation marks. While single and double quotation marks are used for normal strings, triple quotation marks are used for multi-line strings.

Accessing elements of a string

Any character of a string can be accessed using indexing. Python also allows us to use negative indexing to access characters even from the back of a string.

Note: *Indexing of a sequence starts from 0.*

```
# Python program to access characters of a string
my_string = "ScalerTopic"
print("String:", my_string)
# Printing first character of the string.
print("First character:", my_string[0])
# Printing last character of the string.
print("Last character:", my_string[-1])
```

Output:

```
String: ScalerTopic
First character: S
Last character: c
```

In the above example, we are accessing elements of the string using indexing. We are printing the first character of the string using the 0th index and the last character of the string using the -1th index (as Python supports negative indexing).

- **Lists in Python**

Lists are an ordered sequence of one or more types of values. They are just like arrays in other languages. They are mutable, i.e. their items can be modified.

Syntax:

Lists are of the type list. They are created by enclosing items (separated by commas) inside square brackets [].

```
# Python program to demonstrate lists.
my_list = [1, 10, "Scaler", 4, "A"]
print(my_list)
print("Its data type is", type(my_list))
```

Output:

```
[1, 10, 'Scaler', 4, 'A']
Its data type is <class 'list'>
```

In the above example, we are creating a list by enclosing the elements inside square brackets[] and then printing it. In the second line, we are printing its type using the type() function.

Accessing elements of a list

Elements of a list can be accessed by referring to their index numbers, negative indexes to access the list items from its back.

```
# Python program to access elements of a list.
my_list = [1, 10, 'Scaler', 4, 'A']
print("List:", my_list)
# Accessing first element of the list.
print("First element of the list:", my_list[0])
# Accessing 4th element of the list.
print("4th element of the list:", my_list[3])
```

```
# Accessing last element of the list.
print("Last element of the list:", my_list[-1])
```

Output:

```
List: [1, 10, 'Scaler', 4, 'A']
First element of the list: 1
4th element of the list: 4
Last element of the list: A
```

In the above example, we are accessing the elements of the list using indexing. At first, we are accessing the first element of the list by referring to its index "0" (Index number = Position of the element - 1) then we are referring to the fourth element by its index "3" and the last element of the list is accessed using negative indexing.

- **Tuples in Python**

Like lists, Tuples are also an ordered sequence of one or more types of values except that they are immutable, i.e their values can't be modified. They are represented by the tuple class.

Syntax:

Tuples are of tuple type. They are created by a sequence of values separated by a comma with or without parentheses "()" for grouping of the sequence.

```
# Python program to demonstrate tuples.
my_tuple = (1, 10, 'Scaler', 4, 'A')
print(my_tuple)
print("Its data type is", type(my_tuple))
```

Output:

```
(1, 10, 'Scaler', 4, 'A')
Its data type is <class 'tuple'>
```

In the above example, we are creating a tuple by enclosing the elements inside parentheses and then printing it. In the second line, we are printing its type using the type() function.

Accessing elements of a tuple

Elements of a tuple can be accessed by referring to their index numbers, negative indexes to access the tuple items from its back.

```
# Python program to access elements of a tuple.
my_tuple = (1, 10, 'Scaler', 4, 'A')
print("Tuple:", my_tuple)
# Accessing first element of the tuple.
print("First element of the tuple:", my_tuple[0])
# Accessing 3rd element of the tuple.
print("3rd element of the tuple:", my_tuple[2])
# Accessing last element of the tuple.
print("Last element of the tuple:", my_tuple[-1])
```

Output:

```
Tuple: (1, 10, 'Scaler', 4, 'A')
First element of the tuple: 1
3rd element of the tuple: Scaler
Last element of the tuple: A
```

In the above example, we are accessing the elements of a tuple using indexing. At first, we are printing the first element using the 0th index, then we are printing its third element using the 2nd index, at last, we are printing its last element using negative indexing.

3. Dictionaries in Python

In Python, Dictionaries are an unordered collection of key-value pairs (key: value). This helps in retrieving the data and makes the retrieval highly optimized, especially in cases of high volume data. Keys can't be

repeated in a dictionary while values can be repeated. Dictionaries are mutable, i.e. their values can be modified.

Syntax:

Dictionaries are of the type dict. The elements of a dictionary are enclosed in curly brackets {}, where each element is separated by a comma , and key-value pair by a colon :.

```
# Python program for demonstrating dictionaries.
my_dict = {"Name": "Tom", "Age": 50, "Movie": "Mission
Impossible"}
print(my_dict)
print("Its data type:", type(my_dict))
```

Output:

```
{'Name': 'Tom', 'Age': 50, 'Movie': 'Mission Impossible'}
Its data type: <class 'dict'>
```

In the above example, we are creating a dictionary by enclosing the key-value pair in curly brackets {} then we are printing it. In the second line, we are printing its type using the type() function.

Accessing values of a dictionary

Values of a dictionary are accessed by referring to their keys.

```
# Python program for accessing values of a dictionary.
my_dict = {'Name': 'Tom', 'Age': 50, 'Movie': 'Mission
Impossible'}
print(my_dict["Name"])
print(my_dict["Age"])
print(my_dict["Movie"])
```

Output:

```
Tom
50
Mission Impossible
```

In the above example, we are accessing the values of a dictionary by referring to their keys, in the first line we are referring to Tom using the key Name, then we are referring to 50 using the key Age, at last, we are referring to Mission Impossible using the key Movie.

4.Booleans in Python

Boolean is a data type that has one of two possible values, True or False. They are mostly used in creating the control flow of a program using conditional statements.

Syntax:

The True value in the boolean context is called "truthy" and False value is called "falsy".

```
# Python program to demonstrate boolean.
a = True
b = False
# Printing boolean values
print("a =", a)
print("b =", b)
print()
# Data type of True and False
print("Data type of 'True':", type(a))
print("Data type of 'False':", type(b))
```

Output:

```
a = True
```

```
b = False
Data type of 'True': <class 'bool'>
Data type of 'False': <class 'bool'>
```

In the above example, we are printing the boolean variables a (True) and b (False), then we are printing their types using the type() function.

5. Sets in Python

Sets in Python are an unordered collection of elements. They contain only unique elements. They are mutable, i.e. their values can be modified.

Syntax:

Sets are of the type set. They are created by elements separated by commas enclosed in curly brackets {}.

```
# Python program for demonstrating sets.
my_set = {1, 8, "Scaler", "F", 0, 8}
print(my_set)
print("Its data type:", type(my_set))
```

Output:

```
{0, 1, 'F', 8, 'Scaler'}
Its data type: <class 'set'>
```

In the above example, we are creating a set by enclosing the elements inside curly brackets "{}" then we are printing it. In the second line, we are printing its type using the type() function. While creating the set, we have put two duplicate values 8 but when we printed it, 8 occurred only once, that's because sets only have unique elements.

- Data types are classes in Python, whereas variables are objects of these classes.
- Each and every value or variable has a data type in Python.
- Numeric data types in python represent data with numeric values.
- Sequence data types in python are an ordered collection of items.

- Dictionary (also called maps, in other languages) helps in the optimized retrieval of values.
- Boolean has two types of values, True or False, True for truthy values, False for falsy values.
- Sets are an unordered collection of unique elements (like set theory in mathematics).

Expression in Python

A combination of operands and operators is called an **expression**. The expression in Python produces some value or result after being interpreted by the Python interpreter. An expression in Python is a combination of operators and operands.

An example of expression can be : $x=x+10$. In this expression, the first 1010 is added to the variable x. After the addition is performed, the result is assigned to the variable x.

Example :

```
x = 25          # a statement
x = x + 10      # an expression
print(x)
```

Output :

```
35
```

An expression in Python is very different from statements in Python. A statement is not evaluated for some results. A statement is used for creating variables or for displaying values.

Example :

```
a = 25      # a statement
print(a)    # a statement
```

Output :

```
25
```

An expression in Python can contain **identifiers**, **operators**, and **operands**. Let us briefly discuss them.

An **identifier** is a name that is used to define and identify a class, variable, or function in Python.

An **operand** is an object that is operated on. On the other hand, an **operator** is a special symbol that performs the arithmetic or logical computations on the operands. There are many types of operators in Python, some of them are :

- **+ :** add (plus).
- **- :** subtract (minus).
- **x :** multiply.
- **/ :** divide.
- **** :** power.
- **% :** modulo.
- **<< :** left shift.
- **>> :** right shift.
- **& :** bit-wise AND.
- **| :** bit-wise OR.
- **^ :** bit-wise XOR.
- **~ :** bit-wise invert.
- **< :** less than.
- **> :** greater than.
- **<= :** less than or equal to.
- **>= :** greater than or equal to.
- **== :** equal to.
- **!= :** not equal to.
- **and :** boolean AND.
- **or :** boolean OR.
- **not :** boolean NOT.

The **expression** in Python can be considered as a logical line of code that is evaluated to obtain some result. If there are various operators in an expression then the operators are resolved based on their precedence. We have various types of expression in Python, refer to the next section for a more detailed explanation of the types of expression in Python.

Types of Expression in Python

We have various types of expression in Python, let us discuss them along with their respective examples.

1. Constant Expressions

A constant expression in Python that contains only constant values is known as a constant expression. In a **constant expression** in Python, the operator(s) is a constant. A constant is a value that cannot be changed after its initialization.

Example :

```
x = 10 + 15 # Here both 10 and 15 are constants but x is a variable.
print("The value of x is: ", x)
```

Output :

```
The value of x is: 25
```

2. Arithmetic Expressions

An expression in Python that contains a combination of operators, operands, and sometimes parenthesis is known as an **arithmetic expression**. The result of an arithmetic expression is also a numeric value just like the constant expression discussed above. Before getting into the example of an arithmetic expression in Python, let us first know about the various operators used in the arithmetic expressions.

Operator	Syntax	Working
+	x + y	Addition or summation of x and y.
-	x - y	Subtraction of y from x.
x	x x y	Multiplication or product of x and y.
/	x / y	Division of x and y.
//	x // y	Quotient when x is divided by y.
%	x % y	Remainder when x is divided by y.
**	x ** y	Exponent (x to the power of y).

Example :

```
x = 10
y = 5
addition = x + y
subtraction = x - y
product = x * y
division = x / y
power = x**y
print("The sum of x and y is: ", addition)
print("The difference between x and y is: ", subtraction)
print("The product of x and y is: ", product)
print("The division of x and y is: ", division)
print("x to the power y is: ", power)
```

Output :

```
The sum of x and y is:  15
The difference between x and y is:  5
The product of x and y is:  50
The division of x and y is:  2.0
x to the power y is:  100000
```

3. Integral Expressions

An **integral expression** in Python is used for computations and type conversion (**integer** to **float**, a **string** to **integer**, etc.). An integral expression always produces an integer value as a resultant.

Example :

```
x = 10      # an integer number
```

```
y = 5.0     # a floating point number# we need to convert the floating-point number into an integer or vice versa for summation.
result = x + int(y)
print("The sum of x and y is: ", result)
```

Output :

```
The sum of x and y is: 15
```

4. Floating Expressions

A **floating expression** in Python is used for computations and type conversion (**integer** to **float**, a **string** to **integer**, etc.). A floating expression always produces a floating-point number as a resultant.

Example :

```
x = 10      # an integer number
y = 5.0     # a floating point number# we need to convert the integer number into a floating-point number or vice versa for summation.
result = float(x) + y
print("The sum of x and y is: ", result)
```

Output :

```
The sum of x and y is: 15.0
```

5. Relational Expressions

A **relational expression** in Python can be considered as a combination of two or more arithmetic expressions joined using relational operators. The overall expression results in either True or False (boolean result). We have four types of relational operators in Python (i.e.>,<,>=,<=)(*i.e.*>,<,>=,<=).

A relational operator produces a boolean result so they are also known as **Boolean Expressions**.

For example :
10+15>20

In this example, first, the arithmetic expressions (i.e. 10+15 and 20) are evaluated, and then the results are used for further comparison.

Example :

```
a = 25
b = 14
c = 48
d = 45        # The expression checks if the sum of (a and
b) is the same as the difference of (c and d).
result = (a + b) == (c - d)
print("Type:", type(result))
print("The result of the expression is: ", result)
```

Output :

```
Type: <class 'bool'>
The result of the expression is:  False
```

6. Logical Expressions

As the name suggests, a logical expression performs the logical computation, and the overall expression results in either True or False (boolean result). We have three types of logical expressions in Python, let us discuss them briefly.

Operator	Syntax	Working
and	x and y	The expression return True if both x and y are true, else it returns False.
or	x or y	The expression return True if at least one of x or y is True.
not	not x	The expression returns True if the condition of x is False.

Note :
In the table specified above, x*x* and y*y* can be values or another expression as well.

Example :

```
from operator import and_
x = (10 == 9)
y = (7 > 5)
and_result = x and y
or_result = x or y
not_x = not x
print("The result of x and y is: ", and_result)
print("The result of x or y is: ", or_result)
print("The not of x is: ", not_x)
```

Output :

```
The result of x and y is: False
The result of x or y is: True
The not of x is: True
```

7. Bitwise Expressions

The expression in which the operation or computation is performed at the bit level is known as a **bitwise expression** in Python. The bitwise expression contains the bitwise operators.

Example :

```
x = 25
left_shift = x << 1
right_shift = x >> 1
print("One right shift of x results: ", right_shift)
print("One left shift of x results: ", left_shift)
```

Output :

```
One right shift of x results: 12
One left shift of x results: 50
```

8. Combinational Expressions

As the name suggests, a **combination expression** can contain a single or multiple expressions which result in an integer or boolean value depending upon the expressions involved.

Example :

```
x = 25
y = 35
result = x + (y << 1)
print("Result obtained : ", result)
```

Output :

```
Result obtained: 95
```

Whenever there are multiple expressions involved then the expressions are resolved based on their precedence or priority. Let us learn about the precedence of various operators in the following section.

Multiple Operators in Expression (Operator Precedence)

The operator precedence is used to define the operator's priority i.e. which operator will be executed first. The operator precedence is similar to the BODMAS rule that we learned in mathematics. Refer to the list specified below for operator precedence.

Precedence	Operator	Name
1.	( ) [ ] { }	Parenthesis
2.	**	Exponentiation
3.	-value , +value , ~value	Unary plus or minus, complement
4.	/ * // %	Multiply, Divide, Modulo
5.	+ –	Addition & Subtraction
6.	>> <<	Shift Operators

Precedence	Operator	Name
7.	&	Bitwise AND
8.	^	Bitwise XOR
9.	pipe symbol	Bitwise OR
10.	>= <= > <	Comparison Operators
11.	== !=	Equality Operators
12.	= += -= /= *=	Assignment Operators
13.	is, is not, in, not in	Identity and membership operators
14.	and, or, not	Logical Operators

Let us take an example to understand the precedence better :

```
x = 12
y = 14
z = 16
result_1 = x + y * z
print("Result of 'x + y * z' is: ", result_1)
result_2 = (x + y) * z
print("Result of '(x + y) * z' is: ", result_2)
result_3 = x + (y * z)
print("Result of 'x + (y * z)' is: ", result_3)
```

Output :

```
Result of 'x + y + z' is: 236
Result of '(x + y) * z' is: 416
Result of 'z + (y * z)' is: 236
```

Difference between Statements and Expressions in Python

Statement in Python	Expression in Python
A statement in Python is used for creating variables or for displaying values.	The expression in Python produces some value or result after being interpreted by the Python interpreter.
A statement in Python is not evaluated for some results.	An expression in Python is evaluated for some results.

Statement in Python	Expression in Python
The execution of a statement changes the state of the variable.	The expression evaluation does not result in any state change.
A statement can be an expression.	An expression is not a statement.
Example : x=3. **Output :** 3	**Example**: x=3+6. **Output :** 9

Statement in Python

A **Python statement** is an instruction that the Python interpreter can execute. There are different types of **statements in Python** language as Assignment statements, Conditional statements, Looping statements.

The token character NEWLINE is used to end a statement in Python. It signifies that each line of a Python script contains a statement. These all help the user to get the required output.

1. Assignment statements

Assignment statement in python is the statement used to assign the value to the specified variable. The value assigned to the variable can be of any data type supported by python programming language such as integer, string, float Boolean, list, tuple, dictionary, set etc.

Types of assignment statements

The different types of assignment statements are as follows.

- Basic assignment statement
- Multiple assignment statement
- Augmented assignment statement
- Chained assignment statement
- Unpacking assignment statement
- Swapping assignment statement

Basic Assignment Statement

The most frequently and commonly used is the basic assignment statement. In this type of assignment, we will assign the value to the variable directly. Following is the syntax.

```
variable_name = value
```

Where,

- **Variable_name** is the name of the variable.
- **value** is the any datatype of input value to be assigned to the variable.

Example

In this example, we are assigning a value to the variable using the basic assignment statement in the static manner.

```
Open Compiler
variable = "Welcome "
print(variable)print(type(variable))
```

Output

Welcome
<class 'str'>

Example

In this example, we will use the dynamic inputting way to assign the value using the basic assignment statement.

```
Open Compiler
variable = "Welcome "
print(variable)print(type(variable))
```

Output

Enter the value to assign to the variable: Welcome to Tutorialspoint
Welcome
<class 'str'>

Multiple Assignment statement

We can assign multiple values to multiple variables within a single line of code in python. Following is the syntax.

```
v1,v2,……,vn = val1,val2,……,valn
```

Where,

- **v1,v2,……,vn** are the variable names.
- **val1,val2,……,valn** are the values.

Example

In this example, we will assign multiple values to the multiple variables using the multiple assignment statement.

```
Open Compiler
a,b,c,d =10,"python",23.5,True
print(a,b,c,d)
print(type(a))
print(type(b))
print(type(c))
print(type(d))
```

Output

10 python 23.5 True
<class 'int'>
<class 'str'>
<class 'float'>
<class 'bool'>

Augmented assignment statement

By using the augmented assignment statement, we can combine the arithmetic or bitwise operations with the assignment. Following is the syntax.

```
variable += value
```

Where,

- **variable** is the variable name.
- **value** is the input value.
- += is the assignment operator with the arithmetic operator.

Example

In this example, we will use the augmented assignment statement to assign the values to the variable.

```
Open Compiler
a = 10
print("Initial variable value:",a)
a *= 10
print("Augmented assignment variable value:",a)
```

Output

Initial variable value: 10
Augmented assignment variable value: 100

Chained assignment statement

By using the chained assignment statement, we can assign a single value to the multiple variables within a single line. Following is the syntax -

```
v1 = v2 = v3 = value
```

Where,

- **v1,v2,v3** are the variable names.
- **value** is the value to be assigned to the variables.

Example

Here is the example to assign the single value to the multiple variables using the chain assignment statement.

```
Open Compiler
a = b = c = d = "Python Programming Language"
print(a)
print(b)
Print(c)
print(d)
```

Output

```
Python Programming Language
Python Programming Language
Python Programming Language
Python Programming Language
```

Unpacking assignment statement

We can assign the values given in a list or tuple can be assigned to multiple variables using the unpacking assignment statement. Following is the syntax -

```
v1,v2,v3 = [val1,val2,val3]
```

Where,

- **v1,v2,v3** are the variable names.
- **val1,val2,val3** are the values.

Example

In this example, we will assign the values grouped in the list to the multiple variables using the unpacking assignment statement.

```
Open Compiler
a,b,c = ["Tutorial",10,"python"]
print(a,b,c)
print(type(a))
print(type(b))
print(type(c))
```

Output

Tutorial 10 python
<class 'str'>
<class 'int'>
<class 'str'>

Swapping assignment statement

In python, we can swap two values of the variables without using any temporary third variable with the help of assignment statement. Following is the syntax.

```
var1,var2 = var2,var1
```

Where,

- **var1, var2** are the variables.

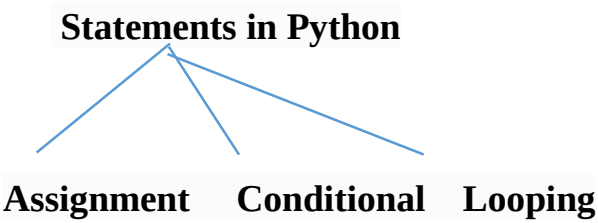
Example

In the following example, we will assign the values two variables and swap the values with each other.

```
Open Compiler
a = 10
b = "Python"
print("values of variables a and b before swapping:",a,b)
a,b = b,a
print("values of variables a and b after swapping:",a,b)
```

Output

values of variables a and b before swapping: 10 Python
values of variables a and b after swapping: Python 10



Tuple assignment:

```
# equivalent to: (x, y) = (50, 100)
x, y = 50, 100

print('x = ', x)
print('y = ', y)
```

OUTPUT

```
x = 50
y = 100
```

When we code a tuple on the left side of the =, Python pairs objects on the right side with targets on the left by position and assigns them from left to right. Therefore, the values of x and y are 50 and 100 respectively.

List assignment:

This works in the same way as the tuple assignment.

```
[x, y] = [2, 4]

print('x = ', x)
print('y = ', y)
```

OUTPUT

```
x = 2
y = 4
```

Sequence assignment:

In recent version of Python, tuple and list assignment have been generalized into instances of what we now call sequence assignment – any sequence of names can be assigned to any sequence of values, and Python assigns the items one at a time by position.

```
a, b, c = 'HEY'

print('a = ', a)
```

```
print('b = ', b)
print('c = ', c)
```

OUTPUT

```
a = H
b = E
c = Y
```

Extended Sequence unpacking:

It allows us to be more flexible in how we select portions of a sequence to assign.

```
p, *q = 'Hello'

print('p = ', p)
print('q = ', q)
```

Here, p is matched with the first character in the string on the right and q with the rest. The starred name (*q) is assigned a list, which collects all items in the sequence not assigned to other names.

OUTPUT

```
p = H
q = ['e', 'l', 'l', 'o']
```

This is especially handy for a common coding pattern such as splitting a sequence and accessing its front and rest part.

```
ranks = ['A', 'B', 'C', 'D']
first, *rest = ranks

print("Winner: ", first)
print("Runner ups: ", ', '.join(rest))
```

OUTPUT

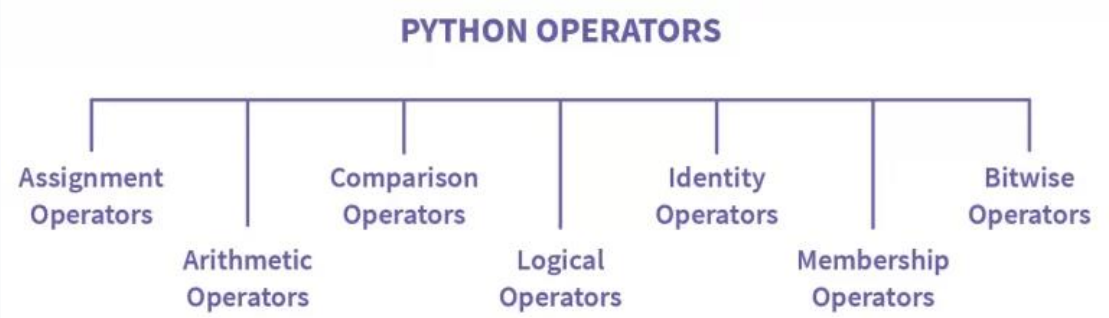
```
Winner: A
Runner ups: B, C, D
```

Operators in Python

Operators in Python are special symbols that carry arithmetic or logical operations. The value that the operator operates on is called the operand. In Python, there are seven different types of operators: arithmetic operators, assignment operators, comparison operators, logical operators, identity operators, membership operators, and boolean operators.

Types of Operators in Python

Operators are divided into 7 categories in Python according to their type of operation.



1. ARITHMETIC OPERATORs in Python

Arithmetic operators are what we have used in our school life. There are 7 types possible under arithmetic operations:

Operator	Operation	Example	Description
+	Addition	a+b	Adds a and b
-	Subtraction	a-b	Subtracts b from a
*	Multiplication	a*b	Multiplies a and b
/	Division	a/b	Divides a and b
%	Modulus	a%b	Gives remainder after dividing a from b
**	Exponential	3**2=9	Ignores the decimal value if present
//	Floor Division	15/2=7	Gives the result of 3 to the power of 2

2. ASSIGNMENT OPERATORs in Python

These are operators used for assigning values to a variable. The values to be assigned must be on the right side, and the variable must be on the left-hand side of the operator. There are various ways of doing it. Let's see each one of them in detail:

Operator	Description	Example
=	Assigns right side operands to left variable.	>>>'number'=10
+=	Added and assign back the result to left operand.	>>>'number'+=20 # 'number'='number'+20
-=	Subtracted and assign back the result to left operand.	>>>'number'-=5
*=	Multiplied and assign back the result to left operand.	>>>'number'*=5
/=	Divide and assign back the result to left operand.	>>>'number'/=2
%=	Taken modulus (Remainder) using two operands and assign the result to left operand.	>>>'number'%=3
**=	Performed exponential (power) calculation on operators and assign value to the left operand.	>>>'number'**=2
**=	Performed exponential (power) calculation on operators and assign value to the left operand.	>>>'number'**=2
**=	Performed floor division on operators and assign value to the left operand.	>>>'number'//=3

3. COMPARISON OPERATORs in Python

These operators compare the value of the left operand and the right operand and return either True or False. Let's consider a equals 10 and b equals 20.

Operator	Description	Example
==	If the values of two operands are equal, then the condition becomes true.	(a==b) is not true.
!=	If the values of two operands are not equal, then the condition becomes true.	(a!=b) is true.
<>	If the values of two operands are not equal, then the condition becomes true.	(a<>b) is true. This is similar to != operator
>	If the values of left operand is greater than the value of right operand, the condition is true.	(a>b) is not true.
<	If the values of left operand is less than the value of right operand, the condition is true.	(a<b) is true.
>=	If the values of left operand is greater than or equal to the the value of right operand, the condition is true.	(a>=b) is not true.
<=	If the values of left operand is less than or equal to the the value of right operand, the condition is true.	(a<=b) is true.

4. LOGICAL OPERATORs in Python

Let's understand the logical operator with an example. You decide you go to school only if your friend also goes. So either you both go to school or you both don't. And that's how an and statement works. If one of the conditions gives False as output, no matter what the other conditions are, the entire statement will return false.

Now in the or statement, if any one of the statements is True, the entire statement returns True. For example, you go to school if your friend comes or you've got a bike. So either of the statements can be True.

The not operator is used to reverse the result, i.e., to make False as True and vice versa.

These operators in Python are used when different conditions are to be met together. There are various types in it:

Operator	Example	Description
and	x < 5 and x < 10	Returns True if both statements are true
or	x < 5 or x < 4	Returns True if one of the statements is true
not	not(x < 5 and x < 10)	Reverse the result, returns False if the result is not true

Let's suppose the value of x is 3, and we do the operation of and on it. It will return True because 3 is less than 5 and 10. For the or operation, it will return True if at least one of the conditions is True. The not operation is an operator which negates the value. For example, if the answer is True, negation is False.

and			or			not	
Statement 1	Statement 1	Result					
True	True	True	True	True	True	True	False
True	False	False	True	False	True	False	True
False	True	False	False	True	True		
False	False	False	False	False	False		

5. IDENTITY OPERATORs in Python

These operators compare the left and right operands and check if they are equal, the same object, and have the same memory location.

Operator	Example	Description
is	x is y	Returns True if both variables are the same object
is not	x is not y	Returns True if both variables are not the same object

Example

Code:

```
number1 = 5
number2 = 5
number3 = 10
print(number1 is number2)  # check if number1 is equal to number2
print(number1 is not number3)  # check if number1 is not equal to number3
```

Output:

```
True
True
```

The first print statement checks if number1 and number2 are equal or not; if they are, it gives output as True; otherwise, False. The next print statement checks for number1 and number3. To return True, number1 and number3 shouldn't be equal.

6. MEMBERSHIP OPERATORs in Python

These operators search for the value in a specified sequence and return True or False accordingly. If the value is found in the given sequence, it gives the output as True; otherwise False. The not in operator

returns true if the value specified is not found in the given sequence. Let's see an example:

Operator	Example	Description
in	5 in {1, 2, 3, 4, 5}	It returns true if it finds the corresponding value in a specified sequence and false otherwise.
not in	5 not in {1, 2, 3, 4}	It returns true if it does not find the corresponding value in a given sequence and true otherwise.

In the first example, 5 is present in the given sequence. Hence it will return True. And in the next example, 5 is not present in the sequence; hence, the not in operator will return True.

7. BITWISE OPERATORs in Python

These operators perform operations on binary numbers. So if the number given is not in binary, the number is converted to binary internally, and then an operation is performed. These operations are generally performed bit by bit.

Let's consider a simple example considering a = 4 and b = 3. At first, both the values are considered in binary format, so 4 in binary is 0100, and 3 in binary is 0011. The bit-by-bit operation is performed when the & operation is performed.

So the last digits of both the numbers are compared, and it returns 1 only if both the numbers are 1; otherwise, 0. Likewise, all the bits are compared one by one and then provide the answer.

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeroes in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

Let's see an example –

Code:

```
number1 = 4  # 0100 in binary
number2 = 3  # 0011 in binary
print("& operation- ", number1 & number2)  # AND
print("| operation- ", number1 | number2)  # OR
print("^ operation- ", number1 ^ number2)  # XOR
print("~ operation- ", ~number2)  # negation
print("<< operation- ", number1 << 1)  # shift left by 1 bit
print(">> operation- ", number1 >> 1)  # shift right by 1bit
```

Output:

```
& operation-  0
| operation-  7
^ operation-  7
```

```
~ operation-  -4
<< operation-  8
>> operation-  2
```

- For the and operation, the results become 0 because no same bit in 4 or 3 is 1.
- In the case of the or operator, if at least one bit is 1, then it returns 1. So here, in the above example, it returns 0111, which is 7.
- If both the bits are the same in the xor operation, it returns 0, otherwise 1. So, here it returns, 0111, that is 7.
- The ~ works on only one operator. It always returns -(n+1), hence -4.
- The << operator shifts the digits but one bit to the left, giving the output as 1000, which is 8. Similarly, the >> operator shifts the bits by one bit to the right side, giving 0010, which is 2, as output.

Operator Precedence in Python

An expression in python consists of variables, operators, values, etc. When the Python interpreter encounters any expression containing several operations, all operators get evaluated according to an ordered hierarchy, called operator precedence.

Python Operators Precedence Table

Listed below is the table of operator precedence in python, increasing from top to bottom and decreasing from bottom to top. Operators in the same box have the same precedence.

Operator	**Description**
:=	Assignment expression
lambda	Lambda expression
if-else	Conditional expression
or	Boolean OR
and	Boolean AND
not x	Boolean NOT
in, not in, is, is not, <, <=, >, >=, !=, ==	Comparisons, membership and identity operators
|	Bitwise OR
^	Bitwise XOR
&	Bitwise AND

Operator	Description
<<, >>	Left and right Shifts
+, –	Addition and subtraction
*, @, /, //, %	Multiplication, matrix multiplication, division, floor division, remainder
+x, -x, ~x	Unary plus, Unary minus, bitwise NOT
**	Exponentiation
await x	Await expression
x[index], x[index], x(arguments…), x.attribute	Subscription, slicing, call, attribute reference
() Parentheses	(Highest precedence)

Python Operators Precedence Rule - PEMDAS

Operator precedence in python follows the PEMDAS rule for arithmetic expressions. The precedence of operators is listed below in a high to low manner.

Firstly, parantheses will be evaluated, then exponentiation and so on.

- **P** – Parentheses
- **E** – Exponentiation
- **M** – Multiplication
- **D** – Division
- **A** – Addition
- **S** – Subtraction

In the case of tie means, if two operators whose precedence is equal appear in the expression, then the associativity rule is followed.

Associativity Rule

All the operators, except exponentiation(**) follow the left to right associativity. It means the evaluation will proceed from left to right, while evaluating the expression.

Example- $(43+13-9/3*7)(43+13-9/3*7)$

In this case, the precedence of multiplication and division is equal, but further, they will be evaluated according to the left to right associativity.

Let's try to solve this expression by breaking it out and applying the precedence and associativity rule.

- Interpreter encounters the parenthesis (. Hence it will be evaluated first.
- Later there are four operators ++, −−, ∗ ∗ and //.
- Precedence of (/,(/, and ∗)>∗)> Precedence of (+,−)(+,−).
- We can use the only operator precedence if all operators belong to different-different levels from the hierarchy table, which is not the case in our example.
- Associativity rule will be followed for operators with the same precedence.
- This expression will be evaluated from left to right, 9/3∗ 79/3∗ 7 = 3∗ 73∗ 7 = 2121.
- Now, our expression has become 43+13−2143+13−21.
- Left to right Associativiy rule will be followed again. So, our final value of expression will be, 43+13−2143+13−21 = 56−2156−21 = 3535.

Comparison Operators

All comparison operations, such as <, >, ==, >=, <=, !=, is, and in, have the same priority, which is lower than arithmetic, shifting, and bitwise operations. Unlike in C, Python follows the conventional mathematical interpretation for expressions like a < b < c.

Comparisons in Python produce boolean values, either True or False.

Python allows chaining of comparisons, such as x < y <= z, which is equivalent to x < y and y <= z. It's important to note that while y is only evaluated once in the chain, z is not evaluated at all **if x < y is found to be false**.

```
a = 100
b = 80
c = 80
d = 19
print(a > b == c >= d)
```

Output:

```
True
```

Explanation: Before evaluating the expression a > b == c >= d, it's important to note that comparison operators in Python have equal precedence and are evaluated from left to right. The expression is transformed based on the comparison chaining rule:

The expression a > b == c >= d is equivalent to (a > b) and (b == c) and (c >= d).

- The expression is evaluated from left to right. The first comparison a > b is performed. Since a is 100 and b is 80, the result is True.
- Next, the second comparison b == c is evaluated. Both b and c have the value 80, so this comparison is also True.
- Finally, the last comparison c >= d is checked. Since c is 80 and d is 19, the condition holds True.
- Evaluating the chained comparisons in order, we have True and True and True, which results in the final output of True.

Note that comparisons, membership tests, and identity tests, all have the same precedence and have a left-to-right chaining feature.

Examples

Below are two examples to illustrate the operator precedence in python. See the explanation to get a good idea of how these things work internally.

1. Precedence of Operators

```
a = (10 + 12 * 3 % 34 / 8)
b = (4 ^ 2 << 3 + 48 // 24)
print (a)
print (b)
```

Output:

```
10.25
68
```

Explanation:

- In the first expression, we have 4 operators: +,∗ ,/,%+,∗ ,/,%.

- Precedence of (/,%(/,% and ∗)>∗)> Precedence of (+)(+).
- The 'associativity rule' will be followed for operators with the same precedence.
- See the transformations shown below to get an understanding of how the entire evaluation will be followed.
- 12 * 3 % 34 / 8 = 36 % 34 / 8 = 2/8 = 0.25
- Now, we have only single operator; hence it would be evaluated simply, 10 + 0.25 = 10.25.
- Hence, the final value is: 10.2510.25.

- In the second expression, we have four operators: XOR, <<,+,//<<,+,//.

  - Precedence of (//)(//) >> Precedence of (+)(+) >> Precedence of (<<)(<<) >> Precedence of (XOR)(*XOR*).
  - After Evaluation of ////, 48//24 = 2 our expression will become (4 ^ 2 << 3 + 2).
  - After Evaluation of ++, 3+2 = 5 our expression will become (4 ^ 2 << 5).
  - After Evaluation of <<<<, 2<<5 = 64 our expression will become (4 ^ 64).
  - After the Evaluation of XOR, 4^64 = 68, our expression will become 68.
  - Hence, the final value is: 6868.

2. Associativity Rule of Operators

```
a = (24 ** 2 // 4 % 25 / 19 * 8)
b = (4 << 8 >> 2)
c = (3 ** 2 ** 4)
print (a)
print (b)
print (c)
```

Output:

```
8.0
256
43046721
```

Explanation:

- Operator precedence in Python is the same for all operators of this example. It is simple to understand that the associativity rule will be followed for evaluation.
- Left to right associativity will be followed in the first two expressions.
- The point worth noticing here is that the exponentiation operator follows right to left associativity.

Comments

Comments in Python are just short descriptions along with the code. It is used to increase the readability of a code.
They are just meant for the developers to understand a piece of code. Python interpreter completely ignores comments in Python.
A single-line comment in Python begins with a hash mark (#) and whitespace character and continues to the line's end.
Let’s take a look at what a good comment and a bad comment looks like –
A bad comment can be any of these things –

- Over-commenting
- Lack of comments
- Misleading comments.

Here are some of them –

```
# misleading/incorrect comment
/* This is called pre-increment
first, increase the value of x
by 1 and then use it
*/
x++;

#obvious comment
#looping in range from 0 to 9 and then printing the value to the console
for i in range(10)
  print(x)
```

On the flip side, a good comment would look like this –

```
# x is incremented by 1 depending on the condition
x++;
```

```
# 0, 1 .. 9
for i in range(10)
  print(x)
```

Via this comparison, you can clearly see which code is more readable, elegant, and helpful.

What are Comments in Python Used For?

Learning to write comments in Python is also a valuable skill. There are several reasons why comments in Python are used. Some of them are-

- Explaining the code.
- Making the code more readable.
- Preventing execution while testing code.

Different Types of Comments in Python

1. Single-Line Comments in Python

The single-line comments in Python are denoted by the hash symbol "#" and include no white spaces. As the name suggests, this kind of comment can only be one line long. They come in handy for small explanations and function declarations.

Check out the following example –

```
# This is a Single-Line comment
# Print "Scaler Academy!" to console
print("Scaler Academy!")
```

2. Multi-Line Comments in Python

Python does not have multi-line comments, but there are two ways we can achieve this –

- You can write "#" at the beginning of every line you wish to comment out. This stimulates multi-line comments in Python. Let's take a look at how we do that –

```
#Welcome
#to
#Scaler
#Academy
```

- For including multi-line comments in Python, we make use of the delimiter (""""). The text to be commented on is enclosed within the delimiter. We mostly use multi-line comments when the text doesn't fit in a single line or when the comment spans more than a few lines. The following code snippet shows how we make multi-line comments using the delimiter-

"""
If you do not want to type # every time for
every line, you can make use of
delimiters!
"""

3. Docstring Comments in Python

Docstring is an in-built feature in Python and is usually associated with documentation in Python. They are added right below the functions, classes, or modules, and they describe what they do. In Python, Docstring is made available using the doc attribute.
The way to implement Docstring in a code is as follows –

```
def intro():
  """
This function prints Welcome to Scaler
  """
  print("Welcome to Scaler")
print(intro.__doc__)
```

The use of Docstring describes what the function does and gives the following output –
This functions prints Welcome to Scaler

Advantages of Using Comments in Python

Consider a scenario where your senior developer has asked you to make some last-minute changes to a particular function in your code. You panic slightly because you don't remember whether you added comments or not. But once you open your codebase, you are relieved because Python comments saved you!
The benefits of comments in Python are endless; hence it's advisable to use comments in Python. Let's take a look at some of them –

- Readability- Python comments greatly enhance the readability of the code. They make the code easy to understand, and there is no need to remember why you wrote a certain code block. So Python comments not only ensure ease when you go back to read your own code from 6 months ago but are also easy for other developers in understanding your code.
- Blocking out certain code- Blocking out code for testing or execution is another avenue where comments are widely used. So

the next time you wish to test out some specific lines of code, you can always comment on the remaining ones.

How to Write Good Comments in Python?

Only writing comments is not enough. Writing feasible and efficient comments is the main objective. So here are some things you can keep in mind to get better at commenting in Python-

- Always use comments to describe what a code block does generically and not in specific detail.
- Write comments only where they are necessary. In other words, get rid of redundant comments that may clutter your codebase and cause confusion.
- Keep them short and simple. This ensures that anyone can read them quickly and not waste too much time trying to decipher them.

Python Functions

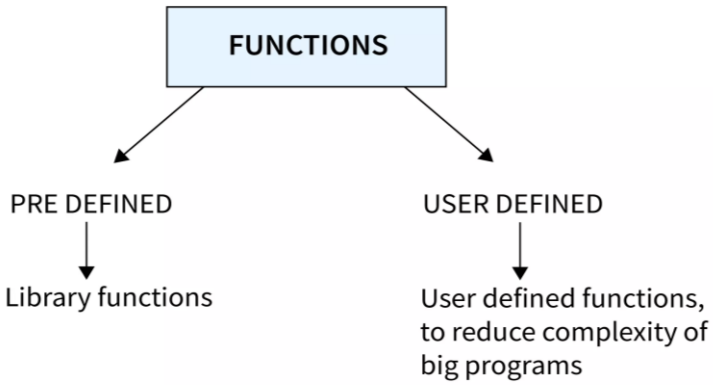
Functions are modular blocks of code designed to perform specific tasks. They enhance code efficiency and clarity by reducing code repetition and enabling code reuse.

Types of Functions in Python

Python primarily categorizes functions into two types:

1. **Built-in Functions:** These are predefined functions in Python that are readily available for use. Examples include len(), range(), and abs().
2. **User-defined Functions:** As the name suggests, these are the functions defined by the users to perform specific tasks.

For a comparative understanding of functions in different programming languages, you can explore the Types of Functions in C.



Built-in Functions

Built-in functions are already defined in python. A user has to remember the name and parameters of a particular function. Since these functions are pre-defined, there is no need to define them again.

Some of the widely used built-in functions are given below:

Example:

```
l = [2, 5, 19, 7, 43]

print("Length of string is",len(l))
print("Maximum number in list is ",max(l))
print("Type is",type(l))
```

Output:

```
Length of string is 5
Maximum number in list is  43
Type is <class 'list'>
```

In the above program, we created a list and stored a few numbers in it, then called various in-built functions on the list and displayed the output.

Some of the widely used python built-in functions are:

Function	Description
len()	Returns the length of a python object
abs()	Returns the absolute value of a number
max()	Returns the largest item in a python iterable
min()	Returns the largest item in a python iterable
sum()	Sum() in Python returns the sum of all the items in an iterator
type()	The type() in Python returns the type of a python object
help()	Executes the python built-in interactive help console
input()	Allows the user to give input
format()	Formats a specified value
bool()	Returns the boolean value of an object

Advantages of functions in Python

- **Helps in increasing modularity of code** – Functions in python help the user to divide the program into smaller parts and solve them individually, thus making it easier to implement.
- **Minimizes Redundancy** – Python functions help us to save the effort of rewriting the whole code. All we got to do is call the function once it is defined.
- **Maximizes Code Reusability** – Once a function is defined in python, it can be called as many times as needed, thus enhancing code reusability.
- **Improves Clarity of Code** – Since a large program is divided into sections with the help of functions, it helps increase the readability of code while ensuring easy debugging.

Python Function Declaration

In Python, a function is declared using the keyword def, succeeded by the name of the function and enclosed parentheses, which may enclose parameters. The syntax is as follows:

```
def function_name(parameters):

        # function body
```

- **Function Name:** This is the identifier for the function. It follows the same naming conventions as variables in Python. The function name should be descriptive enough to indicate what the function does.

- **Parameters (Optional):** These are variables that accept values passed into the function. Parameters are optional; a function may have none. Inside the function, parameters behave like local variables.
- **Function Body:** This block of code performs a specific task. It starts with a colon (:) and is indented. The function body typically contains at least one statement. The return statement can be used to send back a result from the function to the caller. None is returned if no return statement is used.

Syntax of Python Functions

We can create our own functions in Python using the **def** keyword. The syntax of Python functions is given below:

```
def function_name (parameters):
 """docstring"""
   statement_1
   statement_2
   …
   …
    statement_n
     return [expr]
```

Function definition in Python consists of the following components:

- **Keyword def :** It marks the beginning of the function definition.
- **function_name :** It is the name of the function.
- **parameters :** These are the values provided to the function.
- **docstring :** It is used to add the documentation of the function. It is an optional string that explains the functionality of the function.
- **Statements :** These include the sequence of tasks that the function is required to perform. We can also use the **pass** keyword when we don't want to perform any task in the function.
- **Return Statement :** It is an optional statement that is used to return values from the function to the calling code.

Creating a Function in Python

Creating a function in Python involves defining its structure and specifying the code it will execute. This process is straightforward and follows Python's emphasis on readability and simplicity.

Example:

```
def hello():

        print("Hello World")
```

Calling a Function in Python

A function in Python is called by using its name followed by parentheses. In case parameters are present, these are included in the parentheses.

Example: To call the Function named hello, type the following:

```
def hello():
        print("Hello World")
hello()
```

Output:

```
Hello World
```

Need For Function:

- When the program is too complex and large they are divided into parts. Each part is separately coded and combined into single program. Each subprogram is called as function.
- Debugging, Testing and maintenance becomes easy when the program is divided into subprograms.
- Functions are used to avoid rewriting same code again and again in a program.
- Function provides code re-usability
- The length of the program is reduced.

Scope & Lifetime of Variable in Python

In Python, variables act as containers for storing data values. Unlike statically-typed languages such as C/C++/Java, Python doesn't require explicit declaration or typing of variables before use; they are created upon assignment. The variable scope in Python refers to where it can be found and accessed within a program.

Python Local Variables

Local variables in Python are initialized within a function and are unique to that function's scope. They cannot be accessed outside of the function. Let's explore how to define and utilize local variables.

Example

```
def calculate_area(rad):
 pi = 3.14159
 area = pi * rad * rad
print("The area of the circle is:", area)
calculate_area(5)
```

Output:

```
The area of the circle is: 78.53975
```

Flow of Execution

The flow of execution basically refers to the order in which statements are executed. That is to say, execution always starts at the first statement of the program. Moreover, statements execute one at a time. It happens in order, from top to bottom.

Further, functions definitions do not alter the flow of execution of the program. However, it remembers the statements inside the function do not execute until the functions is called.

Moreover, function calls are similar to a bypass in the flow of execution. Thus, in place of going to the next statement, the flow will jump to the first line of the called function. Then, it will execute all the statements there. After that, it will come back to pick up where it left off.

It is essential to remember that when reading a program, do not read it from top to bottom. Instead, keep following the flow of execution. This will ensure that one reads the def statements as they are scanning from top to bottom.

However, one must skip the statements of the function definition until they reach a point where that function is called.

The flow of execution basically refers to the order in which statements are executed. That is to say, execution always starts at the first statement of the program. Moreover, statements execute one at a time. It happens in order, from top to bottom.

If you are a Python developer or enthusiast, understanding the flow of execution of instructions in Python is crucial for optimizing your code and improving its performance. The flow of execution in Python refers to the order in which statements, functions, and modules are executed in a Python program.

This flow can greatly impact the performance of your code, and understanding it can help you identify and fix potential issues. In this tutorial, we will explore the flow of execution in Python and discuss how you can optimize your code to improve its performance. So, let's dive into the world of Python programming and understand the flow of execution in Python!

A function makes dramatic changes in control flow that also affects execution time, readability and reusability of our python program.

When a program is executed and the control can take any statement for execution without following the sequence of written program, and completes the required logic behind it, then this random selection of statements execution is called as Functional flow of execution.

Program without functions

Generally, in a simple and basic python program, you see that every python program is executed in a sequential manner that means line by line,

1. the first line executed firstly,
2. second line executed secondly

and so on to the last line in a Python program but after creating our programs through functions, the functions completely changed its flow of execution. **Now see how.**

Program with functions

When a python program is written through function, then the control is not going to follow the rule of sequential programming,

that means, now the program is not going to be executed line by line (**sequentially**) and provides the flexibility to execute any statement at any time required to fulfil the logic of the program.

Now, instead of going to the next statement, and starting from the first, a python program begins with the top level statements according to the logic of program and the **flow of control** may jumps to the statements required for given logic.

It may also jumps to body of the **function** and executes all the statements there, and then comes back to pick up where it left off. See the diagram for **flow of execution** given below.

If we write a program in Python to add two numbers in a then the structure of the Python program will be like this

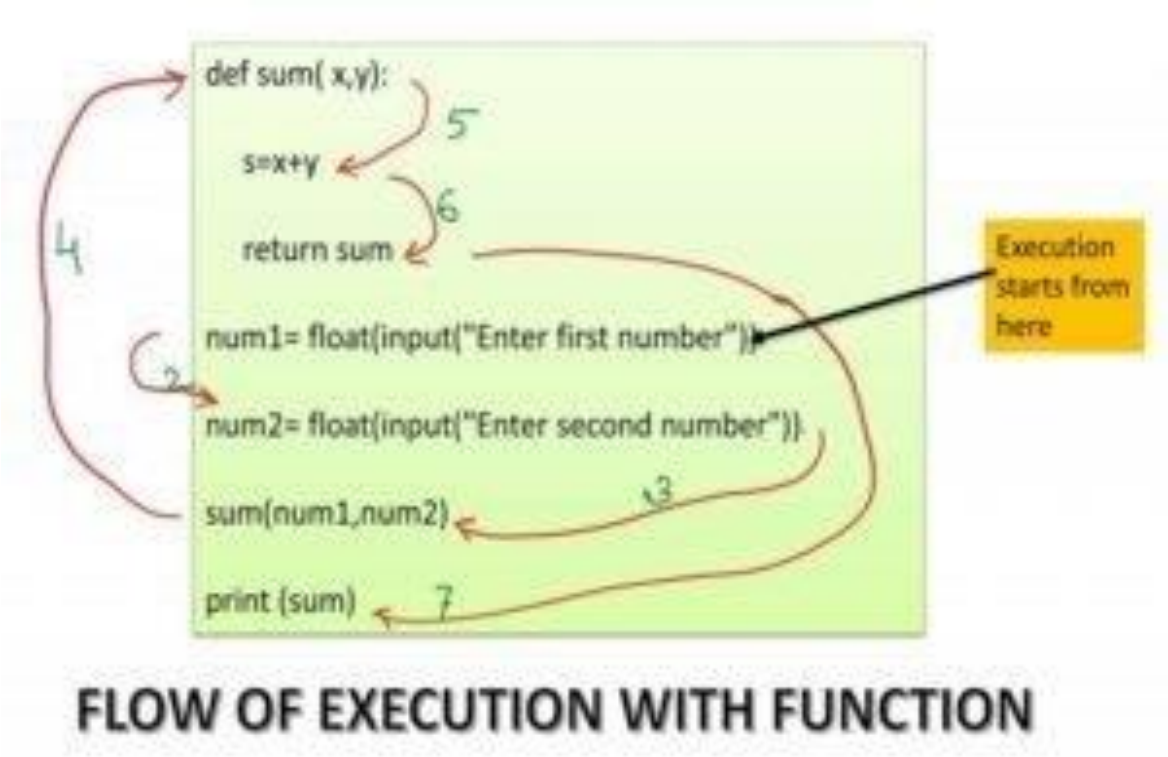
```
num1= float(inpit"Entrr first number"))

num2= float(inpit"Entrr second number"))

sum=num1+num2

print (sum)
```

And , if we write the same program in Python through function then the structure will be like this.

```
def sum( x,y):

    s=x+y

    return sum

num1= float(inpit"Entrr first number"))

num2= float(inpit"Entrr second number"))

sum(num1,num2)

print (sum)
```

Now, compare these steps of execution in both programs.

That means



FLOW OF EXECUTION WITH FUNCTION

Here, you have understood that how the control moves from one line to another following the instruction of program after using functions and **this movement/flow of control is known as flow of execution** of a program.

Python Function Parameters and Arguments

Parameters are variables defined in a function declaration. This act as placeholders for the values (arguments) that will be passed to the function.

Arguments are the actual values that you pass to the function when you call it. These values replace the parameters defined in the function.

Although these terms are often used interchangeably, they have distinct roles within a function.

Parameters

A parameter is the variable defined within the parentheses when we declare a function.

Example:

```
# Here a,b are the parameters
def sum(a,b):
```

```
    print(a+b)
sum(1,2)
```

Output

```
3
```

Arguments

An argument is a value that is passed to a function when it is called. It might be a variable, value or object passed to a function or method as input.

Example:

```
def sum(a,b):

    print(a+b)

# Here the values 1,2 are arguments
sum(1,2)
```

Output

```
3
```

Types of Python Function Arguments

There are **4** inherent function argument types in Python, which we can call functions to perform their desired tasks. These are as follows:

1. **Python Default Arguments**
2. **Python Keyword Arguments**
3. **Python Arbitrary Arguments**
4. **Python Required Arguments**

Default Arguments

As the name suggests,**Default** function arguments in Python are some default argument values that we provide to the function at the time of function declaration.

Thus, when calling the function, if the programmer doesn't provide a value for the parameter for which we specified the default argument, the function assumes the default value as the argument. Thus, calling the function will not result in any error, even if we don't provide any value as an argument for the parameter with a default argument.

We declare a default argument using the equal-to (=) operator at the time of function declaration.

Let us take a look at another example.

```
def print_details(name="Elon Musk", age=50):
        print(f"{name} is {age} years old.")
print_details("Jeff Bezos", 57)
print_details()
O/P
Jeff Bezos is 57 years old.
Elon Musk is 50 years old.
```

In the example above, we specified the values "Elon Musk" and 57 as the default arguments for the parameters' name and age at the time of function declaration.

Then, in line 4, we called the function with user-specified inputs; hence, the output printed statement consisted of the user-specified argument values.

However, in line 5, we did not provide any argument values. Hence, the function printed a statement with the default argument values.

An advantage of default function arguments in Python is that it helps keep the "missing positional arguments" error in check and gives the programmer more control over their code.

Some rules that we need to keep in mind regarding **default** arguments are as follows:

- There can be any number of default arguments in a function.
- Whenever you declare a function, the non-default arguments are specified before the default arguments. The default arguments are

specified at the end, meaning any arguments specified to the right of a default argument must also be a default one. It is to prevent any ambiguity by the Python interpreter. For example, the below-given code block will throw an error if we try to run it.

```
def add_two(a=2, b):
        sum = a + b
        return sum
```

The following is the error we get after running the above-given code block:

```
SyntaxError: non-default argument follows default argument
                                ^
        def add_two(a=2, b):
Line 1  (Solution.py)
```

We specified the default argument (a=0) before the non-default argument (b). The correct way to write this function will be:

```
def add_two(b, a=2):
        sum = a + b
        return sum
```

Keyword Arguments

The term “keyword” is pretty self-explanatory. It can be broken down into two parts— a key and a word (i.e., a value) associated with that key.

To understand keyword arguments in Python, let us first look at the example given below.

```
def divide_two(a, b):
        res = a / b
        return res
res = divide_two(12, 3)
```

```
print(res)  # Output: 4
```

Output:

```
4.0
```

In the above-given example, when calling the function, we provided arguments 12 and 3 to the function. Thus, the function automatically assigned the argument value 12 to parameter a and 3 to parameter b, as per the order/position we specified the arguments. Hence, 12 and 3 here are known as positional arguments.

But this brings us to an important question. What if we wanted the interpreter to assume the argument values as a=3 and b=12 in the above-given example? Can we specify argument b before a during the function call?

Here comes the concept of keyword arguments. The programmer manually assigns argument values to the function call regarding the keyword argument.

The following example shows the use of keyword arguments in Python for the above-defined 'divide_two' function.

```
# both keyword arguments
res = divide_two(a=3, b=12)
print(res)  # Output: 0.25
# one positional, one keyword argument
res2 = divide_two(36, b=12)
print(res2)  # Output: 3 # Both keyword arguments, the order changed
res3 = divide_two(b=12, a=48)
print(res3)  # Output: 4
```

Output:

```
0.25
```

```
3.0
4.0
```

As you can notice, Python allows calling a function using a mixture of both positional and keyword arguments. However, in terms of the order, one must keep in mind that keyword arguments follow positional arguments. Also, a best practice is always to have only positional or keyword arguments in a function call to maintain homogeneity and code readability.

The programmer can use keyword arguments in Python to have stricter control over the arguments passed to the function. It also means that the order of the arguments can be changed, which is not the case in positional arguments, where the order of arguments is according to the order of function parameters, as specified during the function declaration.

Another advantage of using keyword arguments over positional arguments is that it makes the code more human-readable and understandable, which is essential if you are working with a team of developers on a single code base.

Positional Arguments

Positional arguments are called according to their position in the function definition. The first argument in the call corresponds to the first parameter in the function's definition, the second to the second parameter, and so on. This approach is contrasted with keyword arguments, where each argument is assigned to a specific parameter by name, regardless of order.

- Positional arguments make the order of parameters clear, which can be crucial in understanding the function's behaviour.
- They provide a straightforward way to pass values to a function without remembering the parameters' names.

Let us try to understand this with the help of an example.

```
def add(a, b):
        return a + b
result = add(2, 3)
print(result)
```

Output

```
5
```

Here, 2 and 3 are positional arguments.

Variable-length Arguments

Variable length arguments, also known as Arbitrary arguments, play an essential role in Python. Sometimes, at the time of function declaration, the programmer might need to be more specific about the number of arguments to be passed to the function to run it. Another way to put it is that the number of arguments might vary each time the function is called. In these cases, we use arbitrary arguments in Python.

There are two ways to pass variable-length arguments to a python function.

1. Arbitrary Positional Arguments (*args)

The first method is by using the single-asterisk (*) symbol. The single asterisk is used to pass a variable number of non-keyworded arguments to the function. At the time of function declaration, if we use a single-asterisk parameter (e.g., *names), all the non-keyword arguments passed during the function call are collected into a single tuple before being passed to the function. We can understand this with the help of the following example.

```
def print_animals(*animals):     for animal in animals: print(animal) print_animals("Lion", "Elephant", "Wolf", "Gorilla")
```

Output:

```
LionElephant
Wolf
Gorilla
```

2. Arbitrary Keyword Arguments (**kwargs)

You can also pass several keyworded arguments to a Python function. For that, we use the double-asterisk (**) operator. At the time of function declaration, using a double-asterisk parameter (e.g., **address) results in collecting all the keyword arguments in Python, passed to the function at the time of function call into a single Python dictionary before being passed to the function as input. We can understand this with the help of the following example.

```
def print_food(**foods):
        for food in foods.items():
                print(food)
print_food(Lion="Carnivore",Elephant="Herbivore",Gorilla=
"Omnivore")
```

Output:

```
('Lion', 'Carnivore')
('Elephant', 'Herbivore')
('Gorilla', 'Omnivore')
```

The function can also have a combination of arbitrary keyword and non-keyword arguments, as shown in the example below.

```
def print_animal_details(*animals, **foods):
        for animal in animals:
                print(animal)
        for food in foods.items():
                print(food)
print_animal_details("Lion", "Elephant", "Wolf", "Gorilla",
Lion="Carnivore",                    Elephant="Herbivore",
Gorilla="Omnivore")
```

Output:

```
LionElephant
Wolf
Gorilla
('Lion', 'Carnivore')
('Elephant', 'Herbivore')
('Gorilla', 'Omnivore')
```

In the example shown above, we can see that all the non-keyworded function arguments (i.e., “Lion”, “Elephant”, “Wolf”, and “Gorilla”) were collected and stored in the tuple animals. On the other hand, all the keyworded arguments were collected in the dictionary foods.

Modules in Python

Modules in Python are the python files containing definitions and statements of the same. It contains Python **Functions**, **Classes** or **Variables**.Modules in Python are saved with the extension .py.

What are Modules in Python?

Consider a bookstore. In the bookstore, there are a lot of sections like fiction, physics, finance, language, and a lot more, and in every section, there are over a hundred books.

In the above example, consider the book store as a **folder**, the section as python **modules** or files, and the books in each section as python **functions**, **classes**, or python **variables**.

Formal Definition of Python Modules`

Modules in Python can be defined as a Python file that contains Python **definitions** and **statements**. It contains Python code along with Python **functions**, **classes**, and Python **variables**. The Python modules are files with the .py extension.

Features of Python Modules

- Modules provide us the flexibility to organize the code logically. Modules help break down large programs into small manageable, organized files.
- It provides reusability of code.

Example of Modules in Python

Let's create a small module.

- Create a file named example_.py. The file's name is example_.py, but the module's name is example_.
- Now, in the example_.py file, we will create a function called print_name().

Code(example_.py):

```
# We are in the example module
def print_name(name):
        """    This function does not return anything.    It
will just       print your name.    """
        print("Hello! {}, You are in example
module.".format(name))
```

- Now, we have created Python modules example_.
- Now, create a second file at the same location where we have created our example_ module. Let's name that file as test_.py.

Code(test_.py):

```
import example
name = "scaler academy"
example.print_name(name)
```

Output:

```
Hello! scaler academy, You are in the example module.
```

Types of Python Modules

There are two types of Python modules:

1. InBuilt modules in Python
2. User-Defined Modules in Python

InBuilt Module

There are several modules built-in modules available in Python.

Built-In modules like:

- **math**
- **sys**
- **os**
- **random** and many more.

Calling an Inbuilt Module

Code:

```
#calling modules
import math # this is a math module
import random # this is a random module
# now, we use some of the math and random module
function to check whether the modules are working or
not.
cos30 = math.cos(30)
tan10 = math.tan(10)
pie = math.pi
# now, using the random module to generate some
random numbers
random_int = random.randint(0,20)
print(f"Value of cos30 is: {cos30}")
print(f"Value of tan10 is: {tan10}")
```

```
print(f"Value of pie is: {pie}")
print(f"The random number generated using random int function: {random_int}")
```

Output:

```
Value of cos30 is: 0.15425144988758405
Value of tan10 is: 0.6483608274590866
Value of pie is: 3.141592653589793
The random number generated using random int function: 12
```

random.randint() function will generate a new number whenever the function is called. It will generate a new number in the given range. i.e., [0, 20)

Variables in Python Modules

We have already discussed that modules contain functions and classes. But apart from functions and classes, modules in Python can also contain variables in them. Variables like tuples, lists, dictionaries, objects, etc.

Example:

Let's create one more module variables, and save the .py as variables.py

Code:

```
# This is variables module## this is a function in module
def factorial(n):
        if n == 1 || n == 0:
                return 1
        else:
                return n * factorial(n-1)
## this is dictionary in module
```

```
power = {
        1 : 1,
        2 : 4,
        3 : 9,
        4 : 16,
        5 : 25,
        6 : 36,
        7 : 49,
        8 : 64,
        9 : 81,
        10 : 100
}
## This is a list in the module
alphabets = [a,b,c,d,e,f,g,h]
```

Create another Python file, and use the variables module to print the factorial of a number, print the power of 6, and what is the second alphabet in the **alphabet_list**.

Code:

```
import variables
fact_of_6 = variables.factorial(6)
power_of_6 = variables.power[6]
alphabet_2 = variables.alphabets[1]
print(f"The factorial of 6 is : {fact_of_6}")
print(f"Power of 6 is : {power_of_6}")
print(f"Second Alphabet is : {alphabrt_2}")
```

Output:

```
The factorial of 6 is 720
```

```
Power of 6 is 36
The second alphabet is b
```

What is an Import Statement in Python?

The import statement is used to import all the functionality of one module to another. Here, we must notice that we can use the functionality of any Python source file by importing that file as the module into another Python source file.

We can import multiple modules with a single import statement, but a module is loaded once regardless of the number of times it has been imported into our file.

How to Import Modules in Python

For importing modules in Python, we need an import statement to import the modules in a Python file.

Code:

```
import math
print(f"The value of pie using math module is : {math.pi}")
print("The value of pi, we studied is 3.14")
```

Output:

```
The value of pie using the math module is:
3.141592653589793
The value of pi we studied is 3.14
```

Using from <module_name> import <class/function/variable_name> statement

When we are using the import statement to import either built-in modules or user-defined modules, that states we are importing all the functions/classes/variables available in that modules. But if we want to import a specific function/class/variable of that module.

We can import that specific function/class/variable, by using from <module_name> import <class/function/variable_name> statement then we can import that specific resource of that module.

Syntax:

The syntax of from <module_name> import statement is:

```
from <module_name> import <function_name>,
<class_name>, <function_name2>,<variable_name>
```

Example:

Let's see an example of **from <module_name> import <class/function/variable_name>** statement:

Code:

```
from math import sqrt, factorial, pi
print(sqrt(100))
print(factorial(10))
print(pi)
```

Output:

```
10.0
3628800
3.141592653589793
```

Tip:

If we use '*' in place of **function_name/class_name**, it will import all the resources available in that module.

```
from <module_name> import *
# this above statement will import all the available
resources in the module.
```

Example:

Let's see an example of the same:

Code:

```
from math import *
print(pi)
print(sin(155))
print(tan(0))
```

Output:

```
3.141592653589793
-0.8733119827746476
0.0
```

Renaming a Module

Python provides an easy and flexible way to rename our module with a specific name and then use that name to call our module resources. For renaming a module, we use the as keyword. The syntax is as follows:

Code:

```
import math as m
import numpy as np
import random as r
print(m.pi)
print(r.randint(0,10))
print(np.__version__)
```

Output:

```
3.141592653589793
10
```

1.22.0